

Technical and Legal Architectures for Sovereign Proprietary Software: Cryptographic Isolation, Inaudible DRM, and IP Isolation Against Registry Diversion

The Sovereign Software Paradigm in Untrusted Environments

In decentralized and cloud-native computing, software is routinely distributed across third-party infrastructure, exposing intellectual property to virtual machine escapes, hypervisor-level compromises, and supply chain interception. Software sovereignty describes an operational state in which a proprietary application maintains physical intellectual property (IP) isolation, runtime integrity, and cryptographic confidentiality, even when executed on an untrusted or adversarial host. Achieving this requires a synchronized, multi-layered technical and legal framework.

From a technical perspective, software creators must assume that host administrators possess unrestricted root access to physical hardware, operating systems, and memory registers. Consequently, traditional black-box security models are obsolete. Although ideal black-box obfuscation—which guarantees that an adversary can learn nothing from an executable beyond its input-output behavior—is mathematically impossible, modern implementations must target practical alternatives. This includes Indistinguishability Obfuscation ($\mathcal{O}$), which ensures that two semantically equivalent programs are computationally indistinguishable. Because standard $\mathcal{O}$ remains computationally impractical for commercial production, developers deploy white-box cryptography, complex control-flow obfuscation, hardware-isolated execution environments, and silent digital rights management (DRM) telemetry.

These technical layers are co-dependent with modern legal protections. To enforce proprietary rights against overseas piracy, unauthorized scraping, and illegal code distribution, technical architectures must be explicitly mapped to federal statutes and recent court precedents. If technical security "gates" are poorly designed or fail to route cryptographic handshakes through domestic servers, creators may lose their standing to claim trade secret misappropriation, anti-circumvention violations, or unauthorized computer access.

White-Box Cryptography and Algorithmic Shuffling

Mathematical Foundations of White-Box Cryptography

White-box cryptography is engineered for extreme threat models where an adversary has complete visibility and control over execution. The objective is to merge cryptographic keys directly into the compiled application code, making the key indistinguishable from surrounding

operational instructions. The foundational methodology, introduced by Chow, Eisen, Johnson, and van Oorshot (2002/2003), transforms standard symmetric ciphers into a complex network of key-dependent lookup tables. These lookup tables are composed with randomized, invertible linear and non-linear bijective internal encodings to hide the intermediate values computed during encryption.

However, early white-box schemes were broken using structural cryptanalysis, most notably the Billet-Gilbert-Ech-Chatbi (BGE) attack in 2004, which extracts the secret keys by mathematically isolating the boundary encodings of individual rounds. To address structural vulnerabilities, modern architectures utilize incompressible block ciphers, such as SPACE, SPNbox, WhiteKey, or WhiteBlock. These ciphers employ massive, pseudorandomly generated lookup tables that are computationally impossible to compress, thereby neutralizing "code-lifting" attacks where an adversary exfiltrates the entire executable to run it on unauthorized hardware without extracting the raw key.

Security Goal	Definition	Primary Attack Vector	Mitigating Design Pattern
Unbreakability	Absolute resistance to the extraction of the hardcoded secret key.	Algebraic and structural cryptanalysis (e.g., BGE attack).	Random bijective internal encodings and non-linear masking.
Incompressibility	Prevention of compiler or binary size reduction by an adversary.	Code-lifting (unauthorized execution of the intact white-box binary).	Space-hard designs utilizing massive, key-dependent lookup tables.
Traceability	Identification of the source of leaked or redistributed binaries.	Public binary redistribution on file-sharing repositories.	Embedded, hardware-bound digital watermarks and licensing tokens.

Table 1: Strategic Security Goals of White-Box Cryptographic Implementations.

Preventing Passive Leakage via Differential Computation Analysis (DCA)

A major threat to white-box cryptography is Differential Computation Analysis (DCA), which acts as the software counterpart to hardware-based Differential Power Analysis (DPA). Rather than analyzing physical power consumption or electromagnetic emissions, DCA utilizes dynamic binary instrumentation (DBI) frameworks, such as Intel Pin or Valgrind, to record software execution traces containing exact memory addresses and data values written during lookup operations. Because early white-box designs rely on static internal encodings that suffer from mathematical imbalances, the Walsh transform of these encoding functions reveals statistically significant correlations between intermediate values and the secret key.

To counter passive DCA, implementations must combine linear Boolean masking and algorithmic shuffling. Linear masking of order $n-1$ splits every sensitive intermediate variable x in a cryptographic computation into n separate, independent shares:

These shares are processed independently, ensuring that any observation of fewer than n variables yields zero information about the secret x . Algorithmic shuffling randomly permutes the execution order of independent lookup operations, introducing computational noise. For a shuffling degree t , the correlation coefficient score $\text{Cor}(Y, X^*)$ is mathematically reduced: where ρ represents the baseline correlation coefficient of the target variable.

The chronological sequence of these combined protections is highly critical. Applying linear masking and then shuffling is vulnerable to Higher-Order Filtering attacks. To ensure security against algebraic, correlation, and filtering attacks, the compiler must perform Dummy Shuffling *before* applying ISW (Ishai-Sahai-Wagner) masking. This composition is optimized via the S5 (Semi-Shuffled Secret Sharing) masking scheme, which merges shuffling and ISW multiplication within a single runtime structure, limiting memory overhead to approximately 1.44 MB for ciphers such as SM4.

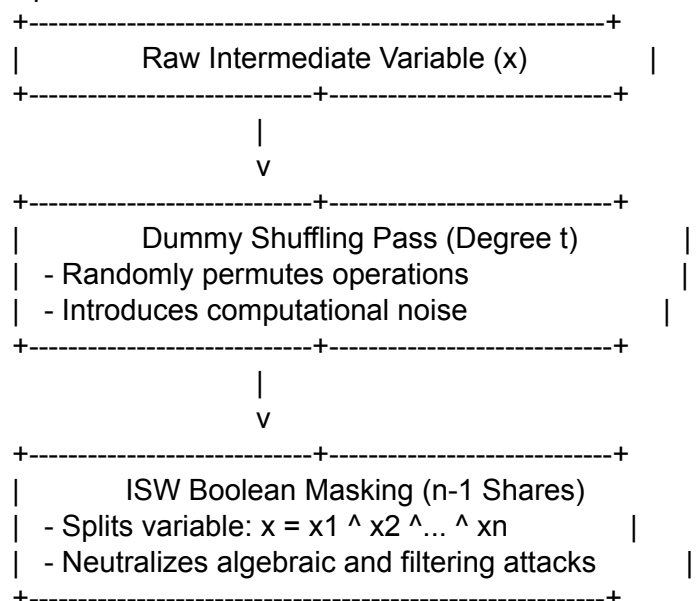


Figure 1: S5 Masking and Shuffling Order of Execution.

Control-Flow Obfuscation and Branchless Security

To prevent static and dynamic analysis of proprietary software logic, developers deploy control-flow obfuscation techniques. Standard compile-time passes (implemented via LLVM transformation passes like Obfuscator-LLVM) modify the code structure through three main patterns:

- **Control-Flow Flattening:** Converts hierarchical loops and nested if-else structures into a single flat state machine managed by a giant loop and switch dispatch structure, obscuring the original logical execution path.
- **Opaque Predicates:** Inserts conditional branch statements whose evaluation is mathematically predetermined at compile time but remains highly complex and seemingly dynamic to static analysis tools. These can be invariant (always evaluating to true or false) or two-way (evaluating to both, requiring semantically equivalent branches).
- **Mixed Boolean-Arithmetic (MBA):** Obfuscates simple algebraic calculations by converting them into equivalent, complex formulations that mix bitwise and arithmetic operations, preventing symbolic execution engines from resolving the underlying equations.

Despite these protections, automated deobfuscation tools leveraging machine learning and binary analysis can detect and remove opaque predicates with up to 98% accuracy. Symbolic and concolic execution engines are also capable of resolving complex path conditions to reconstruct the original control-flow graph (CFG).

To resist these dynamic attacks, developers must transition from branch-based obfuscation to branchless "security from simplicity" architectures. This transformation converts conditional branches into single, linear program segments with no branch statements. By utilizing conditional move instructions and branchless arithmetic, the resulting CFG presents a completely linear topology, neutralizing symbolic solvers and dynamic path analysis.

Silent DRM, Inaudible Watermarking, and Remote Telemetry Architectures

To maintain control over proprietary software deployed in untrusted environments, developers must integrate Digital Rights Management (DRM) and license-compliance telemetry. To prevent detection and removal by reverse engineers, these DRM frameworks are designed to operate "silently" or "inaudibly".

Psychoacoustic DRM and Digital Watermarking

In media-centric software or audio processing engines, "inaudible DRM" refers to acoustic and psychoacoustic digital watermarking. This technique embeds encrypted, low-bitrate digital metadata directly into audio signals, video frames, or compiled resources using psychoacoustic threshold masking. The metadata is mathematically integrated below the threshold of human perception, ensuring it remains completely imperceptible to end-users.

Because the watermark is embedded within the core data stream, it is highly robust. It survives analog leakage, digital compression, file format transcoding, and re-recording. In a sovereign software architecture, this watermark functions as a traitor-tracing token; if a proprietary compiled asset or media output is leaked publicly, the creator can extract the embedded watermark to identify the specific license holder responsible for the breach.

Silent Telemetry and Licensing Enclaves

In standard enterprise software, "inaudible DRM" manifests as silent, background telemetry and license compliance modules. Rather than utilizing disruptive, synchronous licensing checks that alert the user to verification routines, the application compiles silent background threads that execute passively. These threads perform host environmental scans, check the local system against virtualized hardware identifiers, and monitor the validity of licensing files.

This silent DRM paradigm is exemplified by the Cadence License Manager (CLM) and its associated "Phone Home" compliance telemetry. The CLM requires customers to enter a valid, cryptographically signed license file unique to the server's HostID. If an adversary uses a "cracked" version of the software—bypassing the CLM via counterfeit license files—the integrated "Phone Home" system silently detects the tampering. It collects host environmental telemetry (such as usernames, domain directories, IP addresses, and email domains) and transmits this payload back to domestic servers.

To prevent detection by network monitoring tools, the telemetry is designed to be highly resilient. It mimics legitimate background network traffic (such as standard TLS handshakes) and executes asynchronously. If the host environment lacks internet connectivity, the silent DRM caches the encrypted telemetry logs within hidden virtual disks, delaying transmission until connectivity is restored. This covert telemetry allows software creators to accumulate massive,

legally binding databases of unauthorized use without alerting the target of the investigation.

Hardware-Enforced Isolation and Remote Attestation Protocols

When software must execute on untrusted third-party physical servers, software-only obfuscation should be reinforced with hardware-enforced isolation. Trusted Execution Environments (TEEs) partition the processor, memory, and system bus to establish secure execution enclaves.

Process-Based vs. Virtual Machine-Based Isolation

Intel SGX Process Isolation

Intel Software Guard Extensions (SGX) provides process-based isolation by allowing userspace applications to allocate private memory regions called enclaves. SGX implements an extreme trust model that completely excludes the host operating system, hypervisor, system management mode (SMM), and BIOS from the Trusted Computing Base (TCB). The CPU encrypts the physical enclaves inside the Encrypted Page Cache (EPC) using hardware-level memory encryption engines. Although SGX provides a highly granular attack surface, it is historically vulnerable to cache side-channel attacks, such as Foreshadow, SGAXe, and Plundervolt, which manipulate CPU voltage or exploit speculative execution to extract enclave secrets.

AMD SEV-SNP and Intel TDX Virtual Machine Isolation

VM-based TEEs isolate and encrypt entire guest virtual machines, allowing standard operating systems and software stacks to run without modification.

- **AMD SEV-SNP (Secure Nested Paging):** Ephemeral encrypts virtual machine memory using keys managed by the hardware-isolated AMD Secure Processor (ASP). It introduces the Reverse Map Table (RMP) to enforce memory integrity. The RMP tracks page ownership and mappings, preventing the host hypervisor from remapping, copying, replanting, or aliasing memory pages to execute unauthorized modifications.
- **Intel TDX (Trust Domain Extensions):** Allocates isolated "Trust Domains" managed by a hardware CPU module. TDX utilizes Multi-Key Total Memory Encryption (MKTME) to encrypt memory with unique AES-128-XTS keys generated at the silicon level, rendering the memory unreadable to the host hypervisor.

Hardware Architecture	Isolation Level	Excluded from TCB	Memory Encryption Mechanism	Key Vulnerability Profile
Intel SGX	Process-level enclaves.	Operating System, Hypervisor, BIOS, and Firmware.	Encrypted Page Cache (EPC) via hardware AES engines.	Speculative execution and side-channel voltage attacks (e.g., Foreshadow, Plundervolt).

Hardware Architecture	Isolation Level	Excluded from TCB	Memory Encryption Mechanism	Key Vulnerability Profile
AMD SEV-SNP	Guest Virtual Machine.	Bare-metal Hypervisor and external I/O devices.	Ephemeral memory keys managed by the AMD Secure Processor.	Hypervisor-level I/O injection and state rollback if misconfigured.
Intel TDX	Virtual Trust Domain.	Host Hypervisor and management agents.	Multi-Key Total Memory Encryption (MKTME) with AES-128-XTS.	Speculative execution side channels and early-stage VM boot attacks.

Table 2: Hardware-Enforced Trusted Execution Environment (TEE) Comparison.

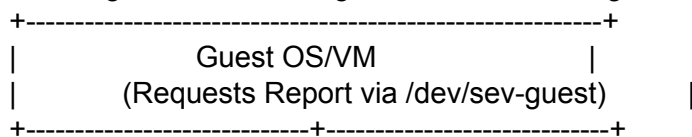
Cryptographic Attestation and Trust Chains

Before provisioning sensitive trade secrets or decryption keys to a remote TEE, a verifier must cryptographically establish that the guest environment is genuine, properly configured, and executing untampered software. This is achieved through remote attestation.

The AMD SEV-SNP Attestation Protocol

In an AMD SEV-SNP environment, the attestation workflow proceeds as follows:

1. **VM Launch and Measurement:** The hypervisor boots the guest VM, and the AMD Secure Processor (ASP) cryptographically measures the initial pages of the virtual firmware (OVMF), generating a unique launch digest.
2. **Generating the Attestation Report:** Once initialized, the guest VM requests an attestation report from the ASP via the `/dev/sev-guest` driver. The VM passes a 512-bit nonce (`REPORT_DATA`) to the ASP to guarantee report freshness and prevent replay attacks.
3. **Hardware-Level Signing:** The ASP aggregates the launch digest, the unique Chip ID, the platform's current firmware Trusted Computing Base (TCB) version, and the user nonce. The ASP signs this report using the Versioned Chip Endorsement Key (VCEK)—a unique, hardware-embedded private ECDSA key derived from the silicon and the TCB version.
4. **Certificate Trust Chain Verification:** The guest VM or verifier retrieves the public certificate chain from the AMD Key Distribution Service (KDS) over the network. The verifier confirms that the VCEK is signed by the AMD SEV Key (ASK), which is signed by the self-signed AMD Root Key (ARK).
5. **Verifying the Signature:** The verifier hashes the attestation report and validates the signature against the VCEK certificate. The extracted measurements are compared against reference "golden" builds of the guest system.



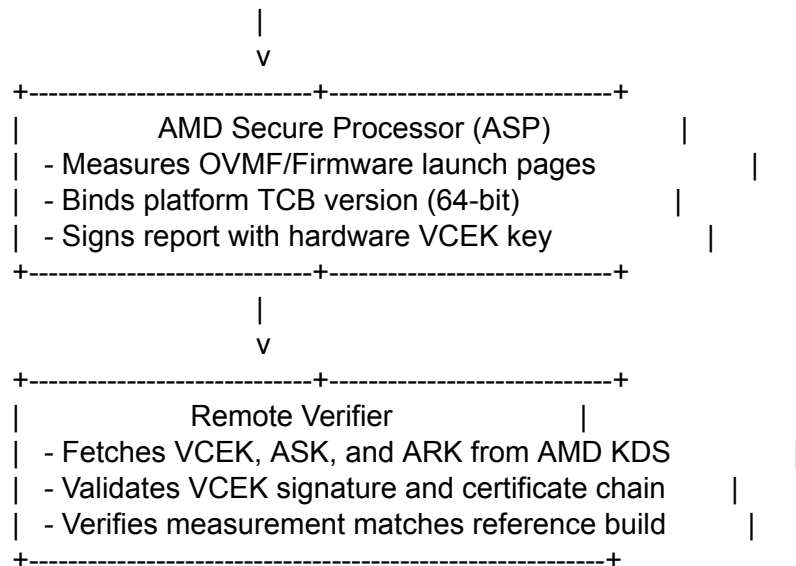


Figure 2: Step-by-Step AMD SEV-SNP Remote Attestation Workflow.

To establish a secure channel after successful attestation, the guest VM must generate ephemeral, self-signed X.509 certificates inside its hardware-isolated boundaries. These ephemeral certificates are linked directly to the TEE-native attestation report, allowing the remote verifier to establish an encrypted TLS tunnel rooted directly in the physical CPU. This architecture prevents man-in-the-middle (MitM) attacks, ensuring that decrypted secrets are accessible only within the isolated memory space of the trusted guest VM.

Supply Chain Protection and Registry Diversion Mitigations

Dependency Confusion Vulnerability Mechanics

Dependency confusion (or namespace confusion) is a supply chain attack vector that targets how modern package managers resolve dependencies across multiple registries. Build systems and developer workstations are frequently configured to pull packages from both a private internal registry (such as JFrog Artifactory or Google Artifact Registry) and a public registry (such as npmjs.com or PyPI).

When a package manager resolves a dependency, its default behavior is to search both registries and automatically select the package with the highest version number. An adversary can discover the names of an organization's internal, private dependencies by crawling public Git repositories, parsing leaked source maps, or reviewing client-side JavaScript bundle files. Once an internal package name is identified, the attacker registers that exact name on the public registry, publishing a malicious version with a highly inflated version number (e.g., 99.9.9). The victim's build pipeline or local workstation then resolves to the malicious public version instead of the internal, private package.

[span_212](start_span)[span_212](end_span)[span_214](start_span)[span_214](end_span)
- internal-auth (v1.0.0)

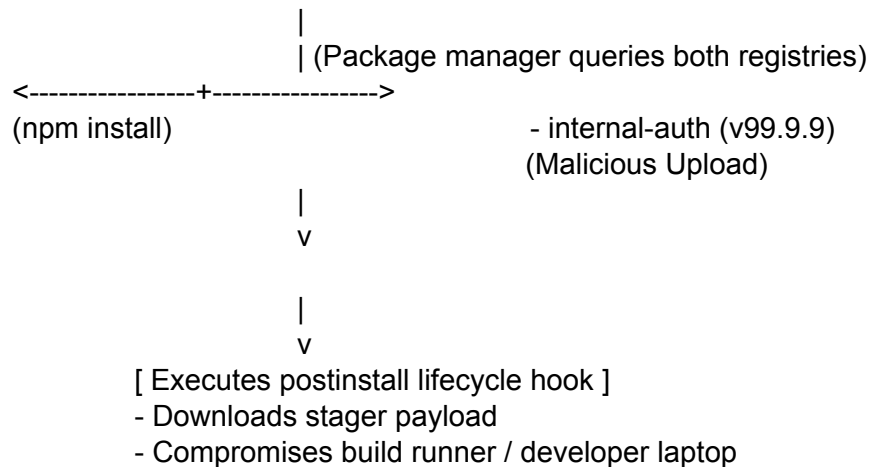


Figure 3: Dependency Resolution Path leading to Compromise.

The primary execution primitive for these attacks relies on package lifecycle hooks, such as NPM's postinstall or preinstall scripts. As documented by Microsoft Threat Intelligence, attackers actively register targeted corporate namespaces (such as `@cloudplatform-single-spa`, `@payments-widget`, and `@sber-ecom-core`). The malicious packages contain fabricated metadata in `package.json`—such as fake GitHub Enterprise repositories and Jira issue trackers—to bypass superficial developer scrutiny.

Upon running `npm install`, the package manager automatically invokes the postinstall hook, executing an obfuscated script. This stager contacts an attacker-controlled command-and-control (C2) endpoint, downloads a platform-specific payload, writes it to a temporary directory, and spawns it as a detached process, compromising the build runner or workstation without requiring the application to import the library.

Hardening the Software Supply Chain

To secure the software supply chain against namespace diversion, organizations must enforce strict scoping, configuration policies, and lockfile validation.

- **Explicit Registry Routing:** Organizations must register their corporate namespaces as scoped packages on public registries (e.g., `@myorg/package`). The local `.npmrc` configuration file must be hardened to map specific scopes exclusively to the private registry, preventing the package manager from ever querying the public registry for internal scopes :

```
# Force all @myorg scoped packages to resolve ONLY via the
[span_204](start_span)[span_204](end_span)private registry
@myorg:registry=https://npm.pkg.github.com/
always-auth=true
```

```
# Set the default registry for standard public packages
registry=https://registry.npmjs.org/
```

- **Disabling Post-Install Scripts:** Because the execution phase of a dependency confusion attack relies on automated lifecycle scripts, CI/CD pipelines must be configured to block script execution during installation. This is enforced by appending the `--ignore-scripts` flag:
`npm[span_205](start_span)[span_205](end_span) install <package-name>`

--ignore-scripts.

- **Enforcing Lockfile Integrity:** Lockfiles (package-lock.json or yarn.lock) do not merely pin version numbers; they store the exact, cryptographically verified SHA-512 hash of every dependency. Build pipelines must enforce strict lockfile execution using the clean installation command: `npm ci`. This command forces the package manager to validate the retrieved package's cryptographic signature against the lockfile. If an upstream public registry attempts to swap a package with a modified version under an identical version number, the cryptographic hash check fails and the build is terminated.
- **PyPI Name Squatting:** Because the Python Package Index (PyPI) lacks native namespace scoping, Python software creators must proactively register (i.e., "squat") their private package names on the public PyPI repository. These public squatted packages must be empty placeholders that contain no proprietary code, configured to immediately throw installation errors upon retrieval to alert developers of configuration leakage.
- **Adapting to NPM 7+ Environment Variables:** Historically, internal software scripts monitored `process.env.npm_config_argv` to inspect arguments passed to the parent NPM CLI. In NPM 7+, this environment variable was removed to prevent environmental exposure and configuration leakage. Modern build scripts must adapt by utilizing the safer `process.env.npm_command` or parsing direct arguments passed after the standard double-hyphen `--` separator in `process.argv`.

Legal Enforcement Frameworks and Jurisprudential Analysis

The deployment of sovereign proprietary software requires an understanding of how technical controls align with federal and international statutes. Technical security design directly dictates legal enforceability.

Extraterritorial Anti-Circumvention (DMCA Section 1201)

The Digital Millennium Copyright Act (DMCA), codified at 17 U.S.C. § 1201, prohibits the circumvention of technological protection measures (TPMs) that effectively control access to copyrighted works. Section 1201 is divided into:

1. **Access Control Provisions:** Prohibits the actual act of circumventing an access-control technology, such as encryption or license managers.
2. **Trafficking Provisions:** Prohibits manufacturing, importing, or distributing devices or tools primarily designed to bypass access controls.

Historically, foreign defendants operating outside the territorial borders of the United States argued that domestic copyright laws and the DMCA did not apply to their activities. However, the federal court ruling in *Cadence Design Systems, Inc. v. Yunjing Intelligent Innovation (Shenzhen) Co.* (N.D. Cal. March 17, 2026) established a critical precedent for the extraterritorial enforcement of Section 1201.

In *Cadence*, a Chinese robotics manufacturer obtained "cracked" versions of Cadence's proprietary electronic design automation (EDA) software from third-party file-sharing websites hosted on foreign servers, bypassing the Cadence License Manager (CLM) using counterfeit license files. To detect unauthorized access, Cadence integrated a "phone home" telemetry module into the compiled software binary, which recorded and transmitted over 200,000 instances of unauthorized use back to Cadence's databases.

The defendants moved to dismiss, asserting that because their physical copying and circumvention occurred in China, U.S. copyright laws could not apply. Under the step-two analysis of the *RJR Nabisco* presumption against extraterritoriality, Judge Araceli Martínez-Olguín held that the focus of Section 1201 is the circumvention of copyright protection systems. Because the authentic license files and key components of the technological security measures were partially created, validated, and maintained on domestic servers located in California, the defendant's use of counterfeit files to bypass those measures constituted a domestic circumvention under the DMCA.

Furthermore, the court established personal jurisdiction under an effects-based theory, ruling that foreign actors who bypass cybersecurity measures maintained on U.S. servers know their actions cause injury to a domestic company. Alternatively, the court ruled that clicking through automated licensing agreements (the SLMA) that contain forum-selection clauses establishes valid consent to jurisdiction.

This case is contextualized by separate regulatory actions against the plaintiff. In an unrelated Department of Justice enforcement action, Cadence pleaded guilty to criminal violations of export controls, paying over \$140 million in penalties for unlawfully exporting sensitive semiconductor design tools to a restricted Chinese military university (the National University of Defense Technology, or NUDT) using front companies. This highlights why technical "phone home" systems are not just commercial DRM, but are legally mandated compliance measures to prevent violating strict export control regimes.

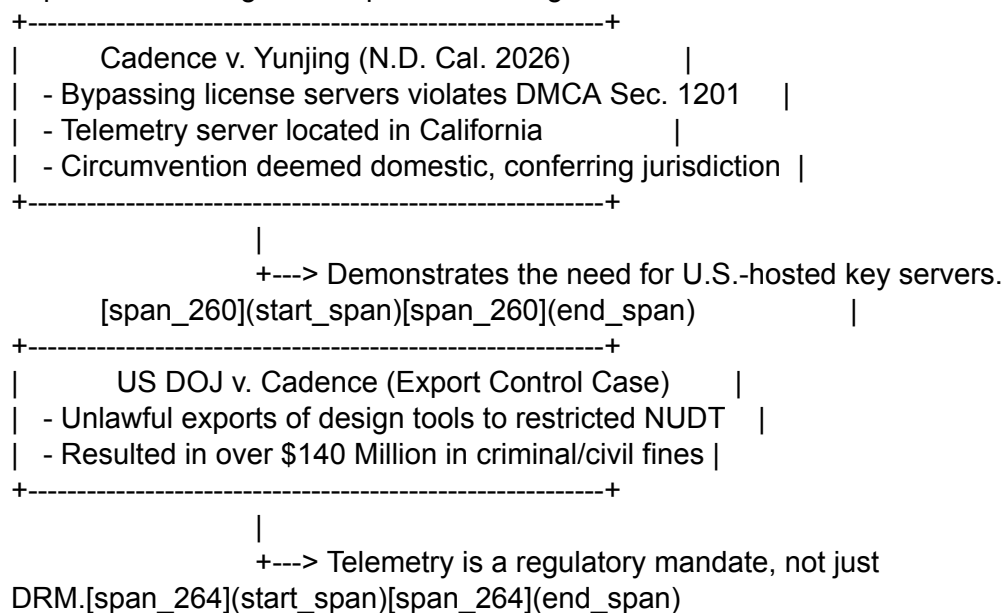


Figure 4: Interconnected Litigation Profile of Cadence Design Systems.

Pleading Standards for Software Trade Secrets (DTSA)

The Defend Trade Secrets Act (DTSA), codified at 18 U.S.C. § 1836, grants software owners a federal civil cause of action for trade secret misappropriation. To secure trade secret protection, a plaintiff must prove that:

1. The software code or algorithm possesses actual or potential independent economic value because it is kept secret.
2. The owner has undertaken "reasonable efforts" under the circumstances to maintain its

secrecy.

Because neither the DTSA nor the Economic Espionage Act (EEA) explicitly defines "reasonable efforts," courts employ a highly fact-specific balancing test, assessing the cost-benefit of the implemented security measures. Defendants frequently seek early dismissal of trade secret claims by asserting that the owner's protective measures were insufficient or poorly implemented.

In the case *Samuel Sherbrooke Corporate Ltd. v. Mayer* (4th Cir. November 18, 2025), the U.S. Court of Appeals for the Fourth Circuit clarified the pleading standard for software trade secrets under the DTSA. The district court had dismissed the plaintiff's trade secret claim, ruling that they failed to allege sufficient physical and digital security measures to protect their proprietary software. The Fourth Circuit reversed, holding that at the pleading stage, a plaintiff is not required to prove absolute secrecy. A complaint plausibly establishes "reasonable efforts" to survive a motion to dismiss if it alleges that the proprietary software was protected by a confidentiality or non-disclosure agreement (NDA) signed by the employee or partner.

While confidentiality provisions are legally sufficient to survive early dismissal, prevailing at trial requires a robust, multi-layered security framework. Sloppy implementation of security protocols can destroy trade secrecy status during discovery. Tested adequate measures for software trade secrets include:

- Requiring signed confidentiality agreements and NDAs for all personnel.
- Enforcing role-based access control (RBAC) and the principle of least privilege, backed by multi-factor authentication and strict access auditing.
- Compiling proprietary software with explicit confidentiality notices, copyright markings, and embedded digital watermarks.
- Segmenting code bases and restricting high-risk source code segments from being downloaded onto personal devices or local storage.
- Utilizing physical security measures, including visitor badges, restricted building access, and the shredding of physical source code printouts.

Actionable Security Level	Legally Adequate Protections (Trial Standard)	Legally Inadequate Security Protections
Pleading Stage (Initial Filing)	Explicit NDA or confidentiality provisions covering the proprietary software.	No signed agreement or contract mentioning the asset.
Legal Status (Trial & Discovery)	Password protection, strict role-based access, and cryptographic logging.	Failing to place confidentiality markings on software materials.
Technical Integration	Eliminating physical access, using MFA, and encrypting source code at rest.	Employing hit-or-miss security (only some key employees signed NDAs).
Procedural Diligence	Requesting courts to file all technical trade secret documents under seal.	Filing trade secret documents with a court without requesting a seal.

Table 3: Legally Adequate vs. Inadequate "Reasonable Measures" under the DTSA.

CFAA Liability Limits and the Gatekeeper Rule

The Computer Fraud and Abuse Act (CFAA), 18 U.S.C. § 1030, imposes criminal and civil liability for accessing a protected computer "without authorization" or "exceeding authorized

access". For decades, proprietary software and website owners utilized the CFAA as a legal tool to block data scrapers, reverse engineers, and competitors from harvesting online data or interacting with web APIs in violation of Terms of Service (ToS) agreements.

This legal strategy was narrowed by the U.S. Supreme Court in *Van Buren v. United States* (2021). In *Van Buren*, the Court adopted a "gates-up-or-down" approach to define unauthorized access. The Court ruled that an individual only "exceeds authorized access" under the CFAA when they have permission to access a computer but proceed to access files, folders, or databases that are off-limits. Violating written policies, employer guidelines, or circumstance-based use restrictions does not violate the CFAA if the system's "gates" are up for that user.

The Ninth Circuit Court of Appeals applied this rule directly to software and data scraping in *hiQ Labs, Inc. v. LinkedIn Corp.* (2022). The court held that because public websites have "no gates to lift or lower," accessing publicly available web data does not constitute access "without authorization" under the CFAA, even if the website owner explicitly revokes permission via a cease-and-desist letter.

This legal reality establishes a critical technical mandate for sovereign software deployment. Since merely violating terms of use or API guidelines does not trigger CFAA liability, a software provider cannot rely on contract terms alone to protect web-facing assets. To invoke the protection of the CFAA, the software owner must implement technical "gates"—such as authenticating firewalls, access keys, or cryptographically validated remote attestation sessions—that are physically lowered. Once a technical gate is lowered, any dynamic bypass, credential reuse, or encryption circumvention constitutes a physical breach of the gate, establishing clear liability under the CFAA.

Platform Liability and Contributory Copyright Infringement

When proprietary software is cracked, redistributed, or hosted on public platforms, the software owner must frequently pursue intermediary platforms (such as Git repositories, content delivery networks, or ISPs) to force removal. However, pursuing secondary liability against platforms was significantly restricted by the U.S. Supreme Court in *Cox Communications, Inc. v. Sony Music Entertainment* (March 25, 2026, No. 24-171).

In *Cox*, music copyright owners sued an internet service provider, asserting that because the ISP had received over 163,000 infringement notices and failed to terminate the accounts of repeat infringers, it was contributorily liable for its users' piracy. The Fourth Circuit affirmed a \$1 billion jury verdict against Cox, holding that supplying a service with knowledge that users will use it to infringe copyrights is sufficient to establish contributory liability.

The Supreme Court reversed, holding that "a company is not liable as a copyright infringer for merely providing a service to the general public with knowledge that it will be used by some to infringe copyrights". The Court ruled that contributory infringement requires proof of intent, which can only be established via two narrow pathways:

1. **Inducement:** The provider actively encourages, markets, or promotes infringement through affirmative conduct.
2. **Tailored Service:** The provider supplies a product or service that is specifically designed or modified for infringement and is not capable of substantial or commercially significant non-infringing uses.

Because Cox provided general internet access (a service with extensive non-infringing uses) and discouraged infringement through its contractual terms, its failure to act on notices was insufficient to establish secondary liability. The Court also clarified that the DMCA's safe harbor

provisions act solely as a shield (defense) and cannot be used as a sword to impose affirmative duties or expand common-law secondary liability.

For sovereign software developers, this ruling establishes that hosting platforms (such as GitHub) or infrastructure providers cannot be held contributorily liable for hosting cracked software or license key generators simply because they have generalized knowledge of infringement. To enforce IP protection successfully, rights holders must move beyond generic notice-and-inaction theories. Enforcement actions must present granular evidence showing user-specific knowledge, active inducement, or systemic practices that actively help the infringement succeed.

Conclusions

The creation and deployment of sovereign proprietary software in adversarial environments requires the integration of cryptographic, hardware, and legal frameworks. Technical security controls must prevent memory inspection, neutralize supply chain registry diversion, and enforce hardware-rooted remote attestation before sensitive assets are decrypted.

Simultaneously, the software deployment model must construct a legal shield that aligns with federal statutes and recent court precedents.

To achieve maximum protection, developers must implement a unified deployment architecture. This includes S5-masked white-box cryptography composed with branchless simplicity algorithms to defend against Differential Computation Analysis and symbolic solvers. It requires hardware-isolated guest environments with remote attestation routed through domestic servers to establish extraterritorial jurisdiction under DMCA Section 1201.

Furthermore, supply chain endpoints must be hardened via explicit package routing scopes, lockfile cryptographic hash enforcement, and disabled post-installation scripts. By binding technical "gates" directly to physical CPU measurements and routing active telemetry verification through domestic infrastructure, software owners establish robust runtime security while maintaining a solid legal foundation to prosecute infringement, trade secret theft, and unauthorized network intrusion globally.

Works cited

1. Intel SGX vs AMD SEV vs ARM TrustZone | Support - Eco, <https://eco.com/support/en/articles/14796363-intel-sgx-vs-amd-sev-vs-arm-trustzone>
2. The Dependency Confusion Attack: How Hackers Slip Into Your Supply Chain - TuxCare, <https://tuxcare.com/blog/the-dependency-confusion-attack-how-hackers-slip-into-your-supply-chain/>
3. Confidential Computing and Attestation: An Introduction to Key Concepts - Medium, <https://medium.com/ultraviolet-blog/confidential-computing-and-attestation-an-introduction-to-key-concepts-35ae25882d6d>
4. Obfuscation through simplicity - University of Auckland, <https://www.math.auckland.ac.nz/~sgal018/simplicity.pdf>
5. On White-Box Cryptography - Marc Joye, <https://marcjoye.github.io/papers/Joy08whitebox.pdf>
6. The ultimate guide to code obfuscation for security professionals - Promon, <https://promon.io/resources/knowledge-center/code-obfuscation-guide>
7. Trade Secrets for Startups: Protection, DTSA, UTSA & Confidential Information (2025), <https://promise.legal/startup-legal-guide/ip/trade-secrets>
8. White-box cryptography - Wikipedia, https://en.wikipedia.org/wiki/White-box_cryptography
9. Everything You Need to Know About White-Box Cryptography - Appknox, <https://www.appknox.com/blog/white-box-cryptography>
- 10.

(PDF) Differential Computation Analysis: Hiding Your White-Box Designs is Not Enough, https://www.researchgate.net/publication/305810868_Differential_Computation_Analysis_Hiding_Your_White-Box_Designs_is_Not_Enough 11. Cadence Design Systems, Inc. v. Yunjing Intelligent Innovation (Shenzhen) Co., Ltd. et al - Justia Dockets, <https://docs.justia.com/cases/federal/district-courts/california/candce/3:2025cv00317/442548/95> 12. Extraterritorial Application of the Digital Millennium Copyright Act ..., <https://tlblog.org/extraterritorial-application-of-the-digital-millennium-copyright-act/> 13. Van Buren v. United States - Wikipedia, https://en.wikipedia.org/wiki/Van_Buren_v._United_States 14. What the Supreme Court Ruling in Cox. v. Sony Means for Tech Providers and Copyright Owners | Insights | Holland & Knight, <https://www.hklaw.com/en/insights/publications/2026/04/what-the-scotus-ruling-in-cox-v-sony-means-for-tech-providers> 15. Scraping Public Websites (Still) Isn't a Crime, Court of Appeals Declares, <https://www.eff.org/deeplinks/2022/04/scraping-public-websites-still-isnt-crime-court-appeals-declares> 16. Fourth Circuit Clarifies 'Reasonable Efforts' Standard for DTSA ..., <https://ipwatchdog.com/2025/12/02/fourth-circuit-clarifies-reasonable-efforts-standard-dtsa-trade-secret-protection/> 17. Unboxing the White-Box Practical Attacks against Obfuscated Ciphers. - Black Hat, <https://blackhat.com/docs/eu-15/materials/eu-15-Sanfelix-Unboxing-The-White-Box-Practical-Attacks-Against-Obfuscated-Ciphers-wp.pdf> 18. Attacks and Countermeasures for White-box Designs? - ORBilu, <https://orбилу.uni.lu/bitstream/10993/33912/1/WhiteBoxAttacksCountermeasures.pdf> 19. A Secure and Efficient White-Box Implementation of SM4 - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11764595/> 20. Differential computation analysis : hiding your white-box designs is not enough - Research portal Eindhoven University of Technology, <https://research.tue.nl/en/publications/differential-computation-analysis-hiding-your-white-box-designs-i/> 21. A White-Box Cryptographic Implementation for Protecting against Power Analysis - J-Stage, https://www.jstage.jst.go.jp/article/transinf/E101.D/1/E101.D_2017EDL8186/_article/-char/en 22. Defeating State-of-the-Art White-Box Countermeasures, <https://d-nb.info/1221417266/34> 23. S5: Combining white-box countermeasures to resist state-of-the-art attacks - 2025 - ORBilu, <https://orбилу.uni.lu/handle/10993/67056> 24. Defeating Opaque Predicates Statically through Machine Learning and Binary Analysis - arXiv, <https://arxiv.org/pdf/1909.01640> 25. A Brief Survey of Code Obfuscation Techniques, <https://chris124567.github.io/2021-06-23-survey-obfuscation/> 26. What is a "control-flow flattening" obfuscation technique?, <https://reverseengineering.stackexchange.com/questions/2221/what-is-a-control-flow-flattening-obfuscation-technique> 27. Trustworthy confidential virtual machines for the masses - arXiv, <https://arxiv.org/html/2402.15277v1> 28. EXPLORING APPROACHES FOR SECURE WORKLOAD DEPLOYMENT AND ATTESTATION IN VIRTUALIZATION-BASED CONFIDENTIAL COMPUTING ENVIRONMENT L - LUTPub, https://lutpub.lut.fi/bitstream/handle/10024/164226/Artemii_Ustiukhin_LUT_Thesis_final.pdf?sequence=1&isAllowed=y 29. Flexible Remote Attestation of Pre-SNP SEV VMs Using SGX Enclaves - IEEE Xplore, <https://ieeexplore.ieee.org/iel7/6287639/10005208/10230233.pdf> 30. Remote attestation overview | Confidential VM | Google Cloud Documentation, <https://docs.cloud.google.com/confidential-computing/confidential-vm/docs/attestation-overview> 31. Confidential Computing: Intel TDX, AMD SEV, and the - Stealth Cloud,

<https://stealthcloud.ai/cloud-paradigms/confidential-computing-deep-dive/> 32. Configuring Confidential Computing with AMD SEV-SNP and VMware ESXi 9.0, <https://lenovopress.lenovo.com/lp2349-configuring-confidential-computing-with-amd-sev-snp-and-vmware-esxi-90> 33. TCG Remote Attestation Protocol | Download Scientific Diagram - ResearchGate, https://www.researchgate.net/figure/TCG-Remote-Attestation-Protocol_fig3_27463209 34. How to perform AMD SEV-SNP attestation inside a guest virtual machine - VPSBG.eu, <https://www.vpsbg.eu/docs/how-to-perform-amd-sev-snp-attestation-inside-a-guest-virtual-machine> 35. AMD SEV usage in YAOOK, <https://docs.yaook.cloud/user/guides/amd-sev/amd-sev-usage.html> 36. sev-snp-utils 1.3.4 - Docs.rs, <https://docs.rs/crate/sev-snp-utils/latest> 37. A TPM-based combined remote attestation method for confidential computing | CNCF, <https://www.cncf.io/blog/2025/10/08/a-tpm-based-combined-remote-attestation-method-for-confidential-computing/> 38. Integrating Remote Attestation with Transport Layer Security - arXiv, <https://arxiv.org/pdf/1801.05863v1/1000> 39. Understanding and Preventing Dependency Confusion Attacks - OX Security, <https://www.ox.security/blog/understanding-and-preventing-dependency-confusion-attacks/> 40. Understanding and Preventing Dependency Confusion Attacks | FOSSA Blog, <https://fossa.com/blog/dependency-confusion-understanding-preventing-attacks/> 41. KingOfBugbounty/Dependency-Confusion-Hunter: Chrome Extension - Passive scanner for Dependency Confusion vulnerabilities in npm/PyPI packages - GitHub, <https://github.com/KingOfBugbounty/Dependency-Confusion-Hunter> 42. Dependency Confusion on npm: Recon via postinstall Hooks Threatens Windows Dev Envs, <https://windowsforum.com/threads/dependency-confusion-on-npm-recon-via-postinstall-hooks-threatens-windows-dev-envs.420857/> 43. node.js - NPM 7 - Pass Flags to Script - Stack Overflow, <https://stackoverflow.com/questions/68061173/npm-7-pass-flags-to-script> 44. [BUG] replacement for npm_config_argv in v8.5.0 · Issue #4663 · npm/cli - GitHub, <https://github.com/npm/cli/issues/4663> 45. Sending command line arguments to npm script - Stack Overflow, <https://stackoverflow.com/questions/11580961/sending-command-line-arguments-to-npm-script> 46. Web scraping, website terms and the CFAA: hiQ's preliminary injunction affirmed again under Van Buren | White & Case LLP, <https://www.whitecase.com/insight-our-thinking/web-scraping-website-terms-and-cfaa-hiqs-preliminary-injunction-affirmed-again> 47. 17 U.S. Code § 1201 - Circumvention of copyright protection systems - Law.Cornell.Edu, <https://www.law.cornell.edu/uscode/text/17/1201> 48. Section 1201: Technology Protection Measures - Copyright Alliance, <https://copyrightalliance.org/education/copyright-law-explained/the-digital-millennium-copyright-act-dmca/section-1201-technology-protection/> 49. Cadence Design Systems Agrees to Plead Guilty and Pay Over \$140 Million for Unlawfully Exporting Semiconductor Design Tools to a Restricted PRC Military University - Department of Justice, <https://www.justice.gov/opa/pr/cadence-design-systems-agrees-plead-guilty-and-pay-over-140-million-unlawfully-exporting> 50. 18 U.S. Code § 1836 - Civil proceedings - Law.Cornell.Edu, <https://www.law.cornell.edu/uscode/text/18/1836> 51. Scraping away at the CFAA | Clifford Chance, <https://www.cliffordchance.com/insights/resources/blogs/talking-tech/en/articles/2021/06/scraping-away-at-the-cfaa.html> 52. Taking Cue from the Supreme Court's Van Buren Decision, Ninth Circuit Releases New Opinion Holding Scraping of Publicly Available Website Data Falls Outside of CFAA - Proskauer,

<https://www.proskauer.com/blog/taking-cue-from-the-supreme-courts-van-buren-decision-ninth-circuit-releases-new-opinion-holding-scraping-of-publicly-available-website-data-falls-outside-of-cfaa> 53. DMCA Takedown Policy - GitHub Docs, <https://docs.github.com/en/site-policy/content-removal-policies/dmca-takedown-policy> 54. Supreme Court Scales Back Copyright Contributory Liability for Online Services - Goodwin, <https://www.goodwinlaw.com/en/insights/publications/2026/03/alerts-practices-supreme-court-scales-back-copyright> 55. Supreme Court Reverses Fourth Circuit in \$1 Billion Copyright Case: What ISPs and Digital Service Providers Need to Know About Secondary Liability After Cox v. Sony Music | Thought Leadership, [https://www.bakerbotts.com/thought-leadership/publications/2026/march/supreme-court-reverses-fourth-circuit-in-\\$1-billion-copyright-case](https://www.bakerbotts.com/thought-leadership/publications/2026/march/supreme-court-reverses-fourth-circuit-in-$1-billion-copyright-case) 56. Supreme Court Narrows Scope of Secondary Copyright Liability in Cox v. Sony, <https://www.kirkland.com/publications/kirkland-alert/2026/04/supreme-court-narrows-scope-of-secondary-copyright-liability-in-cox-v-sony> 57. The Supreme Court Narrows Contributory Copyright Liability for ISPs in Cox v. Sony Music, <https://www.harnessip.com/blog/2026/05/05/copyright-liability-for-isps/> 58. Supreme Court Rejects “Mere Knowledge” Standard for Contributory Copyright Infringement in Cox v. Sony, Reverses \$1 Billion Judgment Against Cox | Crowell & Moring LLP, <https://www.crowell.com/en/insights/client-alerts/supreme-court-rejects-mere-knowledge-standard-for-contributory-copyright-infringement-in-cox-v-sony-reverses-dollar1-billion-judgment-against-cox> 59. Supreme Court Clarifies Limits of Liability for Contributory Copyright Infringement - Finnegan, <https://www.finnegan.com/en/insights/ip-updates/supreme-court-clarifies-limits-of-contributory-liability-for-copyright-infringement.html> 60. Supreme Court Limits Secondary Copyright Liability - Ice Miller, <https://www.icemiller.com/copyright/thought-leadership/supreme-court-limits-secondary-copyright-liability-in-cox-communications-inc-v-sony-music-entertainment> 61. GitHub Acceptable Use Policies, <https://docs.github.com/en/site-policy/acceptable-use-policies/github-acceptable-use-policies>